Edit

Train and Deploy an AI Model in Azure AI Machine Learning Studio

Challenge Overview

Understand the scenario

You are a Developer for Hexelo, an organization that needs to train and deploy an AI model.

In this Challenge Lab, you will train and deploy an AI model in Azure AI Machine Learning Studio. First, you will create a compute instance. Next, you will set up a machine learning job to train and register a model. Finally, you will deploy the model as an online endpoint.

Navigating the Challenge Lab

▶ Understanding a Cloud Slice

▶ Quick tips for navigating the Challenge Lab instructions.

New to Challenge Labs? Click here to learn more.

Create a compute instance

Hints Enabled

No Yes

⚠ Note About Evolving Technology

Azure AI technologies are constantly evolving as Microsoft continues to update, refine, and improve its tools and interfaces. As a result, you may notice slight differences between the instructions in this lab and what you see on your screen. Rest assured, we are actively working to keep all tutorials up to date with the latest features and interfaces. We appreciate your patience and understanding, and we apologize for any inconvenience this may cause.

- Sign into the virtual machine as **Admin** using T Passw0rd! as the password.
- Open **Microsoft Edge**, go to T https://portal.azure.com, sign in to the Microsoft Azure portal as T {USERNAME} using T {PASSWORD} as the password, and then dismiss all prompts.

Expand this hint for guidance on signing into the Microsoft Azure portal.

Want to learn more? Review the documentation on the Azure portal.

- Set up an Azure AI Machine Learning Studio workspace named T HexeloAML-workspace{LAB_INSTANCE_ID} in the **East US** region and the **RG1lod{LAB_INSTANCE_ID}** resource group-for all other properties, use the default.

Expand this hint for guidance on setting up Azure AI Machine Learning Studio.

Want to learn more? Review the documentation on setting up Azure AI Machine Learning Studio.

- Launch Azure Machine Learning Studio from the **HexeloAML-workspace{LAB_INSTANCE_ID}** workspace.

Expand this hint for guidance on launching Azure AI Machine Learning Studio.

Want to learn more? Review the documentation on launching Azure AI Machine Learning Studio.

- Create a **compute** instance named T hexelo-aai-mls.

Expand this hint for guidance on creating a compute instance.

Want to learn more? Review the documentation on creating a compute instance.

⚠ Wait until the Compute instance creation is complete.

- Confirm that the **hexelo-aai-mls** compute instance is in a **Running** state-if not, **Start** the instance.

Expand this hint for guidance on starting a compute instance.

Want to learn more? Review the documentation on starting a compute instance.

Check your work

Verify

Set up a machine learning job to train and register a model

Hints Enabled

No Yes

- Create a new Jupyter **Notebook** named T project1.

  Expand this hint for guidance on creating a Jupyter Notebook.

Want to learn more? Review the documentation on creating a Jupyter Notebook.

- Authenticate **project1.ipynb** to the compute instance to use Azure SDK.

  Expand this hint for guidance on authenticating to the compute instance.

- Obtain your **Subscription ID** and **Current Workspace**, and enter them in the following associated text boxes:

  Subscription ID

  Current Workspace

  Expand this hint for guidance on obtaining your Subscription ID and Current Workspace.

⚠ For the remainder of this Challenge Lab, use the Copy feature to to copy code-do *not* use the Type Text feature to enter your code, as it will introduce formatting errors to the inserted code.

- Create T ml_client for a handle to reference the workspace by using the following code, and then run the code block:

```
from azure.ai.ml import MLClient
from azure.identity import DefaultAzureCredential

# authenticate
```

```
credential = DefaultAzureCredential()

SUBSCRIPTION = "<SubscriptionID>"
RESOURCE_GROUP = "{RESOURCE_GROUP_NAME}"
WS_NAME = "<CurrentWorkspace>"
# Get a handle to the workspace
ml_client = MLClient(
    credential=credential,
    subscription_id=SUBSCRIPTION,
    resource_group_name=RESOURCE_GROUP,
    workspace_name=WS_NAME,
)
```

Expand this hint for guidance on creating a handle to reference the workspace.

Creating MLClient will not connect to the workspace; it will wait for the first time it needs to make a call-this will happen in the next code cell.

Want to learn more? Review the documentation on creating a handle to reference the workspace.

- Add a code cell to **project1.ipynb**, and then add and run the following code:

```
# Verify that the handle works correctly.
# If you get an error here, modify your SUBSCRIPTION, RESOURCE_GROUP, and W
ws = ml_client.workspaces.get(WS_NAME)
print(ws.location, ":", ws.resource_group)
```

Expand this hint for guidance on adding a code cell to a Jupyter notebook.

Want to learn more? Review the documentation on adding a code cell to a Jupyter notebook.

- Create a source folder for the **main.py** training script by adding and running the following code to **project1.ipynb**:

```
import os

train_src_dir = "./src"
os.makedirs(train_src_dir, exist_ok=True)
```

```
import os

train_src_dir = "./src"
os.makedirs(train_src_dir, exist_ok=True)
```

<1 sec

This script handles the preprocessing of the data, splitting it into test and train data.

It then consumes this data to train a tree based model and return the output model. MLFlow will be used to log the parameters and metrics during our pipeline run.

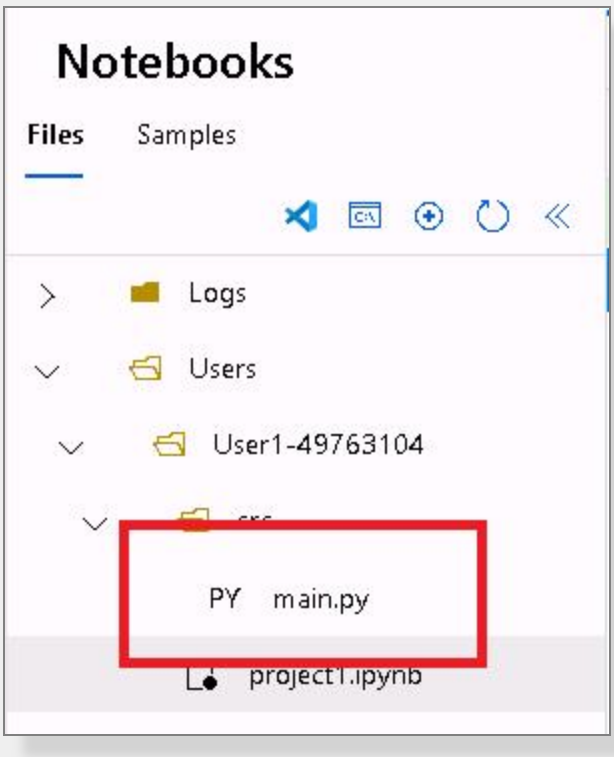
- Write the training script into the directory you just created by adding and running the following code:

```
%%writefile {train_src_dir}/main.py
import os
import argparse
import pandas as pd
import mlflow
import mlflow.sklearn
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

def main():
    """Main function of the script."""

    # input and output arguments
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", type=str, help="path to input data")
    parser.add_argument("--test_train_ratio", type=float, required=False, d
    parser.add_argument("--n_estimators", required=False, default=100, type
    parser.add_argument("--learning_rate", required=False, default=0.1, typ
    parser.add_argument("--registered_model_name", type=str, help="model na
    args = parser.parse_args()

    # Start Logging
    mlflow.start_run()

    # enable autologging
    mlflow.sklearn.autolog()

    ###################
    #<prepare the data>
    ###################
    print(" ".join(f"{k}={v}" for k, v in vars(args).items()))

    print("input data:", args.data)
```

```
    credit_df = pd.read_csv(args.data, header=1, index_col=0)

    mlflow.log_metric("num_samples", credit_df.shape[0])
    mlflow.log_metric("num_features", credit_df.shape[1] - 1)

    train_df, test_df = train_test_split(
        credit_df,
        test_size=args.test_train_ratio,
    )
    ####################
    #</prepare the data>
    ####################

    ##################
    #<train the model>
    ##################
    # Extracting the label column
    y_train = train_df.pop("default payment next month")

    # convert the dataframe values to array
    X_train = train_df.values

    # Extracting the label column
    y_test = test_df.pop("default payment next month")

    # convert the dataframe values to array
    X_test = test_df.values

    print(f"Training with data of shape {X_train.shape}")

    clf = GradientBoostingClassifier(
        n_estimators=args.n_estimators, learning_rate=args.learning_rate
    )
    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_test)

    print(classification_report(y_test, y_pred))
    ###################
    #</train the model>
    ###################

    ##########################
    #<save and register model>
    ##########################
    # Registering the model to the workspace
    print("Registering the model via MLFlow")
    mlflow.sklearn.log_model(
        sk_model=clf,
        registered_model_name=args.registered_model_name,
        artifact_path=args.registered_model_name,
    )

    # Saving the model to a file
    mlflow.sklearn.save_model(
```

```
        sk_model=clf,
        path=os.path.join(args.registered_model_name, "trained_model"),
    )
    ############################
    #</save and register model>
    ############################

    # Stop Logging
    mlflow.end_run()

if __name__ == "__main__":
    main()
```

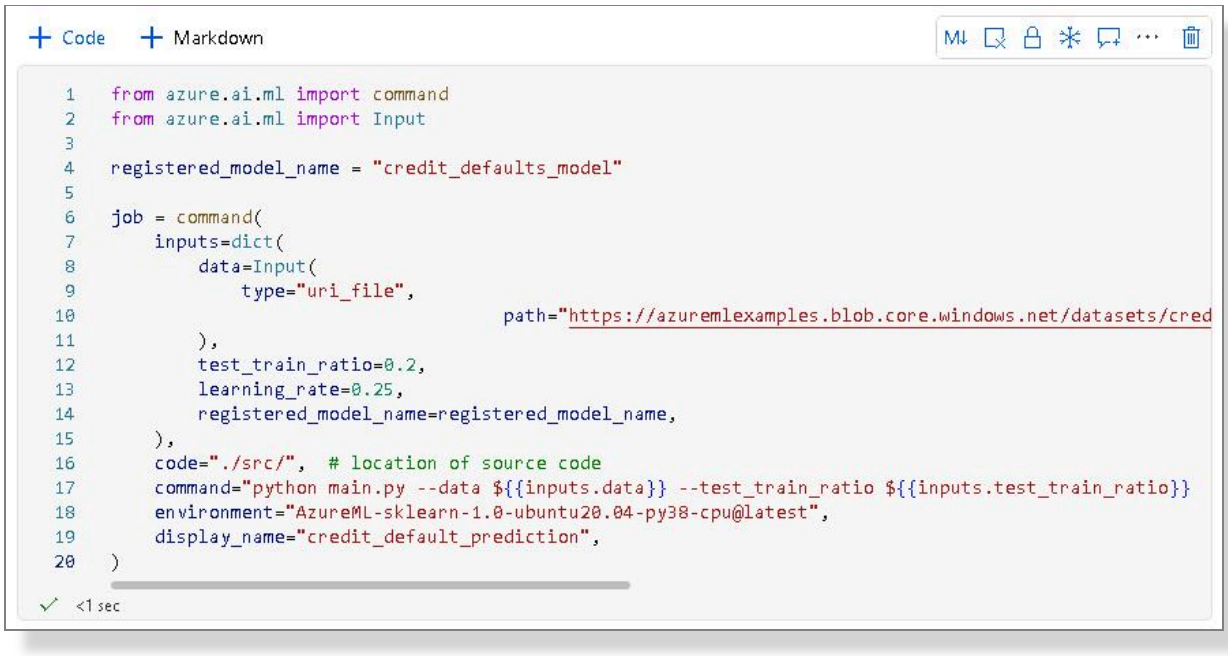
The main.py file can be located in the **src** folder, on the Files tab, in Notebooks. You may need to refresh to view the file.



- Set up a machine learning job to train and register a model for predicting credit card defaults by adding and running the following code:

```
from azure.ai.ml import command
from azure.ai.ml import Input

registered_model_name = "credit_defaults_model"

job = command(
    inputs=dict(
        data=Input(
            type="uri_file",
```

```
                path="https://azuremlexamples.blob.core.windows.net/dataset
        ),
        test_train_ratio=0.2,
        learning_rate=0.25,
        registered_model_name=registered_model_name,
    ),
    code="./src/",  # location of source code
    command="python main.py --data ${{inputs.data}} --test_train_ratio ${{i
    environment="AzureML-sklearn-1.0-ubuntu20.04-py38-cpu@latest",
    display_name="credit_default_prediction",
)
```



Want to learn more? Review the documentation on setting up a machine learning job to train and register a model.

- Submit the job by using the following code:

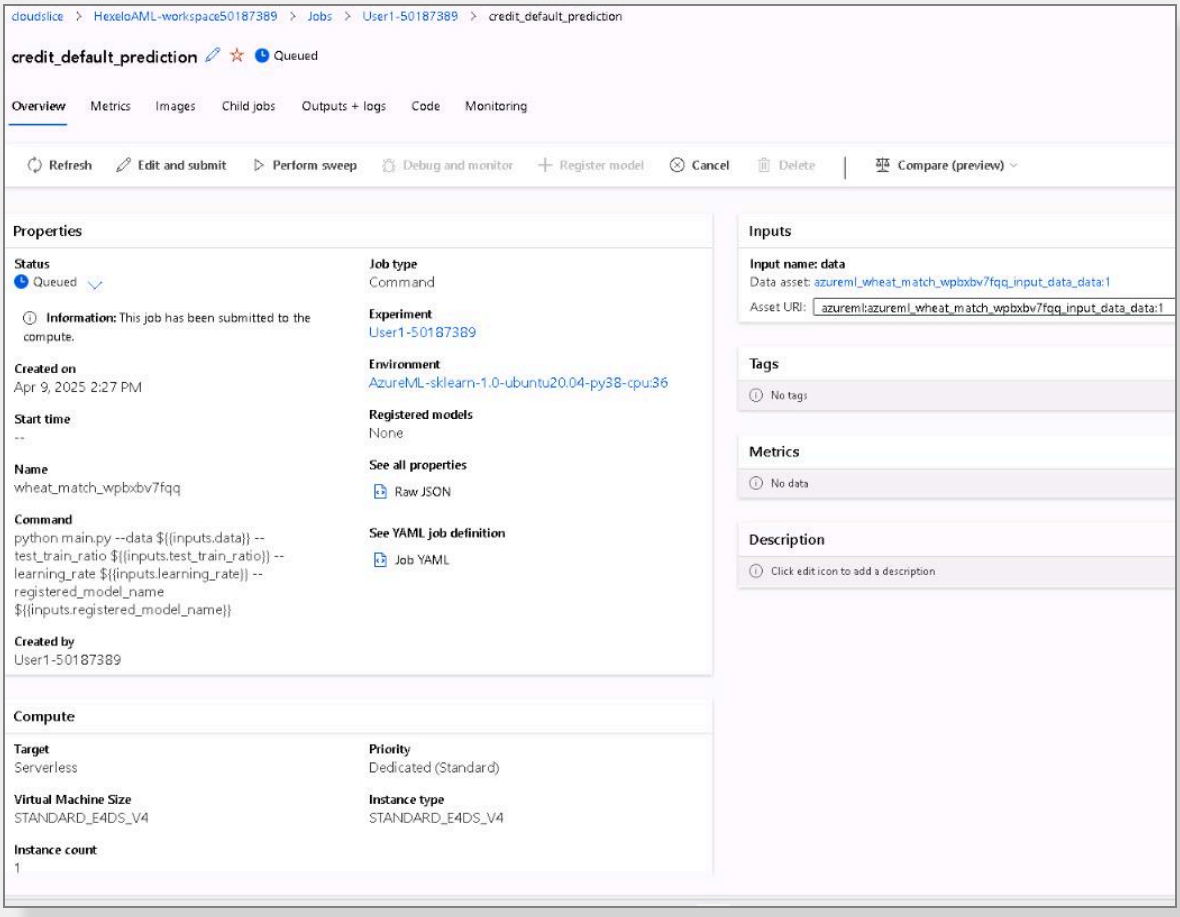
```
ml_client.create_or_update(job)
```

```
1  ml_client.create_or_update(job)
✓ 5 sec

Class AutoDeleteSettingSchema: This is an experimental class, and may change at any time. Please see
https://aka.ms/azuremlexperimental for more information.
Class AutoDeleteConditionSchema: This is an experimental class, and may change at any time. Please see
https://aka.ms/azuremlexperimental for more information.
Class BaseAutoDeleteSettingSchema: This is an experimental class, and may change at any time. Please see
https://aka.ms/azuremlexperimental for more information.
Class IntellectualPropertySchema: This is an experimental class, and may change at any time. Please see
https://aka.ms/azuremlexperimental for more information.
Class ProtectionLevelSchema: This is an experimental class, and may change at any time. Please see
https://aka.ms/azuremlexperimental for more information.
Class BaseIntellectualPropertySchema: This is an experimental class, and may change at any time. Please
see https://aka.ms/azuremlexperimental for more information.
Uploading src (0.0 MBs): 100%|          | 3139/3139 [00:00<00:00, 136953.84it/s]
```

- View job output and wait for job completion.

Expand this hint for guidance on viewing the job output.

The output of this job will look like this in the Azure Machine Learning Studio.



Explore the tabs for various details like metrics, outputs etc. Once completed, the job will register a model in your workspace as a result of training.

⚠ Wait until the status of the job is complete before returning to the notebook to continue. The job will take 2 to 3 minutes to run. It could take longer (up to 10 minutes) if the compute cluster has been scaled down to zero nodes and custom environment is still building.

Check your work

Verify

Deploy the model as an online endpoint

Hints Enabled

No Yes

- Create a new online endpoint in **project1.ipynb** named T UUID by adding and running the following code:

```
import uuid

# Creating a unique name for the endpoint
online_endpoint_name = "credit-endpoint-" + str(uuid.uuid4())[:8]
```

- In **project1.ipynb**, create the endpoint by adding and running the following code:

```
from azure.ai.ml.entities import ManagedOnlineEndpoint
import uuid

online_endpoint_name = "credit-endpoint-" + str(uuid.uuid4())[:8]

endpoint = ManagedOnlineEndpoint(
    name=online_endpoint_name,
    description="Credit prediction endpoint",
    auth_mode="key"
)

ml_client.begin_create_or_update(endpoint).result()

print(f"✅ Created endpoint: {online_endpoint_name}")
```

Expect the endpoint creation to take a few minutes.

- Add the Import by adding and running the following code:

```
from azure.ai.ml.entities import ManagedOnlineDeployment
```

```
model = ml_client.models.get(name=registered_model_name, version=latest_mod

blue_deployment = ManagedOnlineDeployment(
    name="blue",
    endpoint_name=online_endpoint_name,
    model=model,
    instance_type="Standard_DS3_v2",
    instance_count=1,
)

blue_deployment = ml_client.begin_create_or_update(blue_deployment).result(
print(f"✅ Deployment 'blue' completed at endpoint: {online_endpoint_name}"
```

- Retrieve the endpoint by adding and running the following code:

```
endpoint = ml_client.online_endpoints.get(name=online_endpoint_name)

print(
    f'Endpoint "{endpoint.name}" with provisioning state "{endpoint.provisi
)
```

```
1  endpoint = ml_client.online_endpoints.get(name=online_endpoint_name)
2
3  print(
4      f'Endpoint "{endpoint.name}" with provisioning state "{endpoint.provisioning_state}" is retrieved
5  )
```

✓ <1 sec

```
Endpoint "credit-endpoint-ebfa844b" with provisioning state "Succeeded" is retrieved
```

- Select the latest version of the model by using the following code:

```
# Let's pick the latest version of the model
latest_model_version = max(
    [int(m.version) for m in ml_client.models.list(name=registered_model_na
)
print(f'Latest model is version "{latest_model_version}" ')
```

```
1  # Let's pick the latest version of the model
2  latest_model_version = max(
3      [int(m.version) for m in ml_client.models.list(name=registered_model_name)]
4  )
5  print(f'Latest model is version "{latest_model_version}" ')
```

✓ <1 sec

```
Latest model is version "1"
```

- Deploy the latest version of the model by adding and running the following code:

```
# picking the model to deploy. Here we use the latest version of our regist
model = ml_client.models.get(name=registered_model_name, version=latest_mod
```

```
# Expect this deployment to take approximately 6 to 8 minutes.
# create an online deployment.
# if you run into an out of quota error, change the instance_type to a comp
# Learn more on https://azure.microsoft.com/en-us/pricing/details/machine-l
blue_deployment = ManagedOnlineDeployment(
    name="blue",
    endpoint_name=online_endpoint_name,
    model=model,
    instance_type="Standard_DS3_v2",
    instance_count=1,
)

blue_deployment = ml_client.begin_create_or_update(blue_deployment).result(
```

Expect this deployment to take approximately 6 to 8 minutes.

- Deploy the model to the endpoint by adding and running the following code:

```
deploy_dir = "./deploy"
os.makedirs(deploy_dir, exist_ok=True)
```

- Create a sample request file following the design expected in the run method in the score script by adding and running the following code:

```
%%writefile {deploy_dir}/sample-request.json
{
  "input_data": {
    "columns": [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22]
    "index": [0, 1],
    "data": [
            [20000,2,2,1,24,2,2,-1,-1,-2,-2,3913,3102,689,0,0,0,0,689,0,0,0
            [10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        ]
  }
}
```

- Test the deployment by adding and running the following sample data:

```
# test the blue deployment with some sample data
ml_client.online_endpoints.invoke(
    endpoint_name=online_endpoint_name,
    request_file="./deploy/sample-request.json",
    deployment_name="blue",
)
```

Check your work

Verify

Clean up your resources

- Clean up your resources by adding and running the following code:

```
ml_client.online_endpoints.begin_delete(name=online_endpoint_name)
```

Summary

Congratulations, you have completed the **Train and Deploy an AI Model in Azure AI Machine Learning Studio** Challenge Lab.

You have accomplished the following:

- Created a compute instance.
- Set up a machine learning job to train and register a model.
- Deployed a model as an online endpoint.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

⚠ Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to Challenge Labs feedback.

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Manage Azure Resource Deployment by Using an Azure Resource Manager Template [Guided]
- Manage Azure Resource Groups [Guided]
- Create an Azure Function App [Guided]
- Create an Azure Logic App [Guided]
- Configure an Azure Distributed Denial of Service Protection Plan [Guided]
- Manage Encryption by Using an Azure Key Vault [Guided]
- Configure Azure Role-Based Access Control [Guided]

- Configure an Azure Lock [Guided]
- Configure Monitoring by Using Azure Monitor [Guided]
- Deploy an Azure Virtual Machine [Guided]
- View Azure Service Health Options [Guided]
- Manage Microsoft Entra Users and Groups [Guided]
- Implement a Network Security Group [Guided]
- Azure Cost Management [Guided]